Homework 1: DDPM Implementation

CMU 10-799: Diffusion & Flow Matching

Spring 2026

Due: Saturday, January 24, 2026 at 11:59 PM ET **Total Points:** 80 + 20

Late Due Date: Monday, January 26, 2026 at 11:59 PM ET
Submission: Gradescope https://www.gradescope.com/courses/1207241
Starter Code: https://github.com/KellyYutongHe/cmu-10799-diffusion/
Note: *By popular demand, Q6 is now an extra credit question*

Introduction

Welcome to your first homework in CMU 10799 Diffusion & Flow Matching!

In this homework, you'll implement a Denoising Diffusion Probabilistic Model (DDPM) [Ho et al., 2020] from scratch and train it to generate realistic face images. But more than just "getting it to work," we want you to develop the intuition and debugging skills that researchers use every day. The structure of this homework mirrors a pseudo research workflow:

1. Understand your data: Before writing any model code, explore what you're trying to generate
2. Build intuition: Use the 1D playground to see diffusion in action (optional but recommended)
3. Implement: Write the core DDPM algorithm and U-Net architecture
4. Debug: Learn to diagnose common failure modes
5. Experiment: Run ablations to understand design choices
6. Reflect: Document what you learned and what you'd try next

This homework is AI-friendly. You may use any AI coding assistants, chatbots, or reference implementations. You may also use any other resources that you can find on the Internet. At the end of the homework, you'll document what resources you used.

Part 0: Setup your codebase (0 point)

First let's get started and setup your environments and codebase by forking and cloning the startup code repository: https://github.com/KellyYutongHe/cmu-10799-diffusion.

```
git clone https://github.com/KellyYutongHe/cmu-10799-diffusion.git
cd cmu-10799-diffusion

# Run setup script (auto-detects your hardware)
./setup-uv.sh     # Recommended (faster)

# Activate environment
source .venv-*/bin/activate
```

What You'll Implement:

File	What to implement
`src/data/celeba.py`	Data preprocessing transforms
`src/methods/ddpm.py`	Full DDPM algorithm
`src/models/unet.py`	U-Net architecture using provided blocks
`configs/`	Create your own model configs
`train.py`	Add sampling for logging
`sample.py`	Add your sampling scheme

What We Provide:

- **U-Net** [Ronneberger et al., 2015] **building blocks** (`blocks.py`): ResBlock, AttentionBlock, SinusoidalPositionEmbedding, Downsample, Upsample, etc
- **Training infrastructure**: Mixed precision, gradient clipping, checkpointing, logging
- **EMA**: Exponential moving average for stable sampling
- **Evaluation**: KID computation via torch-fidelity [Obukhov et al., 2020]
- **Modal integration**: Cloud GPU training for those without local GPUs
- **Notebooks**: Skeleton notebook for exploration
- **Scripts**: Example scripts to run your training and evaluation jobs on a slurm cluster or on Modal

Compute Budget:

You have **$500 in Modal credits** for the entire course (all 4 homework). Budget wisely!

Part I: Understanding Your Data (10 points)

Since the goal for generative modeling is to model the distribution of your data, it is important to first understand what does this distribution look like.

The dataset for this course is a custom subset of CelebA [Liu et al., 2015], filtered to have specific properties. Your first task is to discover what those properties are.

Q1. Visual Exploration [5 pts]

(a) [2 pts] Visualize a grid of at least 16 random samples from the training set. Include this grid below.

(c) [3 pts] This dataset was filtered from full CelebA using their 40 binary attributes. Based on your visual exploration, hypothesize which attributes were likely used to create this subset. What filtering criteria were used to create this subset? Why?

From the sample grid, I do not think the subset is filtered to contain only one specific demographic group. There are both male and female faces, multiple hair colors, different ages, and different expressions. However, the images are mostly clean, centered portraits with visible faces and little occlusion. My hypothesis is that the filtering removed hard cases such as hats, glasses, strong occlusions, extreme poses, or images where the face is not clearly visible. The goal seems to be a cleaner face-generation subset rather than a subset defined by a single semantic attribute.

Q2. What did you learn? [5 pts]

(a) [3 pts] Imagine you found a pre-trained diffusion model that can reach SOTA performance on full CelebA (200K diverse faces). Now you generated samples and compared them to this filtered subset using KID or FID. Would you expect the FID and KID score to still be SOTA? Why?

I would not necessarily expect the FID or KID score to remain SOTA. A pre-trained model that performs well on full CelebA is trying to match the broad distribution of all CelebA faces, which includes much more variation in pose, occlusion, accessories, image quality, and facial attributes. This filtered subset appears to be a narrower distribution of cleaner, centered, less occluded face images.

FID and KID compare distributions of image features, not just whether each generated image looks realistic by itself. Therefore, even high-quality samples from a full-CelebA model may include valid CelebA faces that are outside the filtered subset distribution. In that case, the

generated distribution $p_\theta(x)$ may be close to the full CelebA distribution but still mismatched with the subset distribution, causing worse FID/KID scores against this subset.

(b) [2 pts] Based on your exploration, what kind of data augmentation transformation do you plan to use for your training and why?

I plan to use only mild data augmentation, mainly random horizontal flipping. The images are aligned face crops, and a horizontally flipped face should still be a realistic sample from the same face distribution. This can increase the effective diversity of the training data without changing the important facial attributes too much.
I would avoid strong rotations, vertical flips, heavy cropping, or aggressive color jitter. Those transformations could create unrealistic faces or shift the training distribution away from the filtered CelebA subset that the diffusion model is supposed to learn.

Part II: Implement DDPM (25 points)

Now it's time to build your first diffusion model! Take a good look at the starter code and start building! Remember, you are free to add, delete and modify any and all parts of the starter code to fit your preference.

Q3. Building Intuition [optional, 0 pts]

Before diving into the full scale training, it is generally a good idea to build some intuitions of the algorithm with some toy examples as they are easier to debug.

(a) [0 pts] We have prepared a Jupyter notebook *01_1d_playground.ipynb* for you to experiment with 1D toy data. This notebook contains some options of mixture of Gaussians and their visualizations, you can try out your algorithm first in this playground.

(b) [0 pts] Full scale training can be difficult to debug sometimes, and that's why in *train.py* we also provide an argument flag, *--overfit-single-batch*, for you to toggle an experiment where you overfit to a single batch of data for sanity checking. You should be able to iterate your model a lot faster with this experiment than full scale training.

Q4. Implementation [25 pts]

Now implement DDPM for real! The starter code provides the skeleton, and your job is to fill in the core algorithm. Remember: this is your codebase. Feel free to modify any part of the starter code to fit your preferences. Add helper functions, reorganize files, change the config structure... whatever helps. The only requirement is that your final code runs and produces good results.

The starter code provides a detailed guideline on which files are provided and which files are the ones that need your implementation. Go check it out!

(a) [5 pts] Train your DDPM model to convergence. You may use either the provided configs or your own. Report:

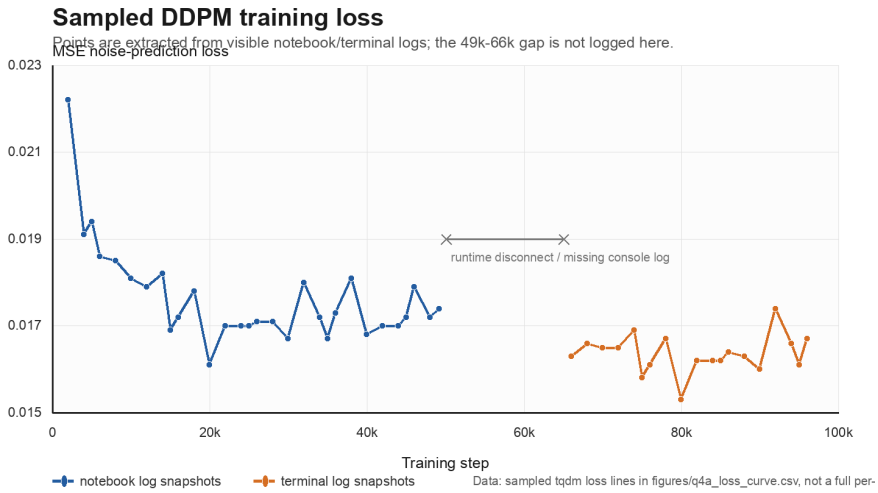
1. Model size
2. Batch size

3. Total training iterations
4. Training loss curve
5. Compute cost (GPU hours)

I trained the DDPM on the provided CelebA 64x64 subset (`electronickale/cmu-10799-celeba64-subset`) using the Colab config in `configs/ddpm_colab.yaml`. The model is a timestep-conditioned U-Net with **15,605,635** parameters ($\approx$ 15.61M).

Batch size	64
Training iterations	100,000
Image shape	$3 \times 64 \times 64$
DDPM timesteps	1000
Beta schedule	linear, 10^{-4} to 2×10^{-2}
Optimizer	AdamW, learning rate 2×10^{-4}
EMA decay	0.9999
Mixed precision	enabled
Final checkpoint	`ddpm_final.pt`

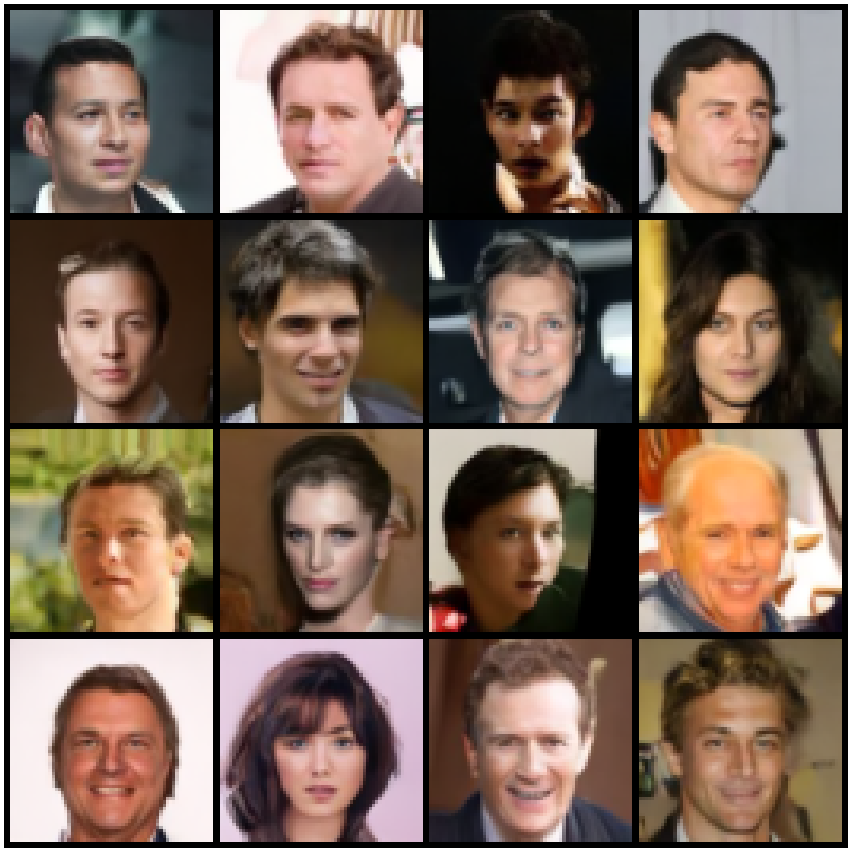
Training was interrupted by Colab runtime disconnects and resumed from saved checkpoints. The checkpoints restore model, optimizer, EMA, AMP scaler, step, and config state. The run used approximately **2.6 GPU-hours** for the actual training loop, based on the checkpoint timestamps and tqdm wall-clock output: roughly 1.5 GPU-hours for steps 0–60k and roughly 1.1 GPU-hours for steps 60k–100k. This estimate excludes package installation, dataset download, manual reconnect time, and post-training evaluation.



The plotted trace is a sampled training-loss trace rather than a full per-step metric log. The points come from the visible tqdm lines in the Colab notebook and terminal output, and the gap between the first and resumed sessions is left explicitly unconnected in the figure. The denoising MSE drops quickly in the first few thousand steps and then stays mostly around 1.6×10^{-2} to 1.8×10^{-2}. The extracted points are saved in `figures/q4a_loss_curve.csv`.

(b) [10 pts] Generate a grid of 16 samples from your trained model. Include this grid below.

The following grid was generated from the final EMA checkpoint after 100,000 training iterations.



(c) [10 pts] Evaluate your model KID with 1k samples and report your KID score (both mean and std). You should be able to obtain KID < 0.005 with single L40S GPU training for a few hours.

I evaluated the final EMA checkpoint using 1000 generated samples and 1000 real images from the CelebA subset. The evaluation used `torch-fidelity` with 10 KID subsets of size 100. The generated and real image counts were both 1000.

$$\mathrm{KID}_{\mathrm{mean}} = 0.006615829467773437, \qquad \mathrm{KID}_{\mathrm{std}} = 0.001997329867895109.$$

This is slightly above the suggested 0.005 target, but it is in the same order of magnitude and the generated samples are visually face-like. The metric was computed from the checkpoint `ddpm_final.pt` with the command path `scripts/evaluate_kid.py`; the JSON output was written to `/content/kid_eval/kid_metrics.json` in the Colab runtime.

(d) [0 pts] Provide a zip file of your code through Gradescope "Homework 1 Code" assignment. Make sure your code is runnable because we may run it to verify your results, and do not include any large files such as model checkpoints or dataset files. If you do not provide a zip file for your code, you will receive 0 point on Part II and Part IV of this homework.

The code zip should include the runnable repository code and configs, but should **exclude** generated datasets, logs, samples, and large checkpoints such as `ddpm_final.pt`. I keep the trained model and generated figures outside the code zip and submit them only through the written report or separate artifact workflow if requested.

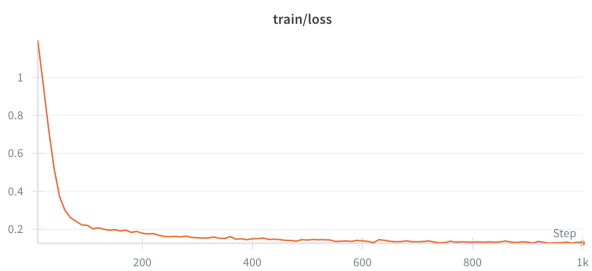
Part III: Help Your "Classmate" Debug (15 points)

Your classmate Kale is having some trouble getting her model to work. As someone who has successfully trained a DDPM model, you decide to help!

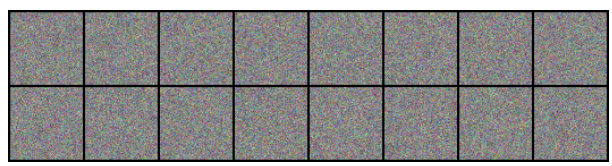
In each of the following scenario, you may be provided a loss curve, a grid of samples or a description of the situation. Try your best to help Kale with her implementation!

Q5. Pseudo Debugging Scenarios [15 pts]

(a) [5 pts] Kale trained a model for 1000 iterations and observed this seemingly nice looking loss curve. However, all her samples are pure noise. What could be the problem here? List at least two potential sources of bugs.



(a) The loss curve she observed



(b) The final samples she obtained

Figure 1: The loss curve and the samples that Kale observed.

A decreasing training loss does not guarantee that the sampling code is correct. During training, the model is only optimized to predict the Gaussian noise ϵ from x_t and t:

$$\mathcal{L} = \mathbb{E}_{x_0,t,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right].$$

Thus, Kale's loss curve can look reasonable even if the reverse sampling path is broken.
One likely bug is an incorrect reverse-process formula. In DDPM sampling, the predicted noise should be used to compute

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right).$$

If the sign, coefficient, posterior variance, or timestep indexing is wrong, the sampler may fail to remove noise even though the training MSE decreases.
Another likely bug is incorrect timestep handling in the sampling loop. Sampling should start from $x_T \sim \mathcal{N}(0, I)$ and iterate backward from $T-1$ to 0. If the loop runs in the wrong

direction, has an off-by-one error, or passes the wrong timestep tensor to the model, the U-Net will denoise under a noise level different from what it saw during training.
I would also check whether the training and sampling schedules match exactly, including β_t, α_t, $\bar{\alpha}_t$, and the number of timesteps. A mismatch means timestep t has different meanings during training and sampling. Finally, Kale should verify that the sampling script loads the correct trained checkpoint and applies EMA weights if those were used for evaluation. Although 1000 iterations may be too short for high-quality face generation, completely pure-noise samples strongly suggest a sampling, schedule, timestep, or checkpoint-loading bug.

(b) [5 pts] Kale observed that after a while her loss would stay at a non-zero level, and it is quite "bumpy" – the loss does not decrease monotonically but instead has small fluctuations. Should she be worried about this? Why does this "bumpiness" exist? Can training for longer still be valuable for improving the model performance in her case?

She should not be worried just because the loss is non-zero and bumpy. The DDPM training objective is an expectation over the data, the timestep, and the sampled Gaussian noise:

$$\mathcal{L} = \mathbb{E}_{x_0,t,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right].$$

Each optimization step only estimates this expectation with one minibatch, one random timestep per image, and one fresh noise tensor. Therefore, the plotted per-step loss is a noisy stochastic estimate, not the exact full objective. It is normal for it to fluctuate locally rather than decrease monotonically.
The loss also does not need to converge to zero. The model has finite capacity, the data distribution is complex, and different timesteps have different difficulty. Very noisy large-t examples contain little visible image structure, while small-t examples require fine local denoising. Because a batch mixes images, timesteps, and random noise, the loss can remain at a positive plateau even when the model is learning useful denoising behavior.
Training longer can still be valuable as long as the loss is not exploding or becoming NaN and the samples or evaluation metrics continue to improve. Even if the scalar MSE changes slowly, additional training may improve high-level face structure, texture details, sample diversity, EMA weights, and KID/FID. Kale should look at a moving average of the loss together with sample grids and distribution metrics instead of judging the model from raw per-iteration loss alone.

(c) [5 pts] After the model fully converged, Kale obtained these samples shown below. What is her bug and how to fix it?

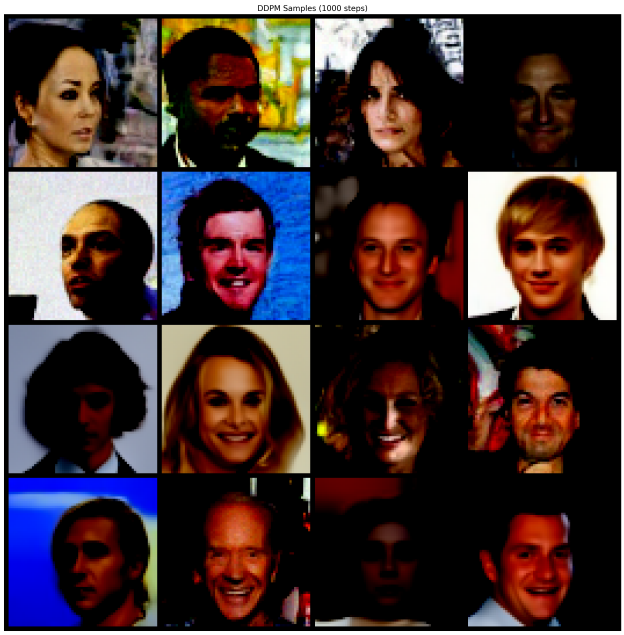


Figure 2: The samples that Kale obtained.

The samples already contain recognizable faces, so the denoising model is not completely broken. The main issue is that the saved images have an incorrect value range: large regions are clipped to black and the colors are overly contrasty. This is the typical symptom of saving tensors from the training range $[-1, 1]$ as if they were display images in $[0, 1]$.
Since the training images are normalized before entering the DDPM, the generated samples should be unnormalized before visualization:

$$x_{\text{display}} = \frac{x_{\text{sample}} + 1}{2}, \qquad x_{\text{display}} = \text{clip}(x_{\text{display}}, 0, 1).$$

If Kale skips this conversion, all negative pixel values are clipped to zero by the image-saving pipeline, which explains the dark backgrounds and saturated appearance.
The fix is to apply the inverse normalization before saving or displaying samples, for example `images = ((samples + 1) / 2).clamp(0, 1)`, or to use a helper such as `unnormalize(samples).clamp(0, 1)`. She should check the visualization code before retraining, because this bug can make a successfully trained model look much worse than it actually is.

Part IV: Ablation Study (10 + 20 points)

In this part of the homework, we will explore different design choices of the DDPM algorithm and how they can affect the final performance.

Q6. Alternative Parametrization [optional extra credits, 20 pts]

Note: By popular demand, this question is now an option extra credit question. I.e. You don't have to do it if you don't want to. The homework now have a total of 80 points instead of 100. If

you still complete that question you will receive 20 points extra credit.

(a) [7 pts] The original DDPM paper parametrized the model to predict the noise ϵ. However, this is not the only option! First derive an alternative parametrization for the same model (i.e. what else can the model predict that can still yield the same mathematical formulation of the algorithm).

The forward process can be written as

$$x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

The original DDPM parameterization trains the network to predict ϵ. This is convenient because once we have $\hat{\epsilon}_\theta(x_t, t)$, we can estimate

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\hat{\epsilon}_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}},$$

and then use this estimate inside the reverse mean.
An equivalent alternative is to train the model to predict x_0 directly:

$$y_\theta(x_t, t) = \hat{x}_{0,\theta}(x_t, t), \qquad y = x_0.$$

This still gives the same DDPM reverse formulation because the corresponding noise prediction can be recovered by rearranging the forward-process equation:

$$\hat{\epsilon}_\theta(x_t, t) = \frac{x_t - \sqrt{\bar{\alpha}_t}\hat{x}_{0,\theta}(x_t, t)}{\sqrt{1-\bar{\alpha}_t}}.$$

Another common alternative is the velocity parameterization

$$v_t = \sqrt{\bar{\alpha}_t}\epsilon - \sqrt{1-\bar{\alpha}_t} x_0.$$

If the network predicts $\hat{v}_\theta(x_t, t)$, then both x_0 and ϵ can be recovered by the linear inverse:

$$\hat{x}_{0,\theta} = \sqrt{\bar{\alpha}_t} x_t - \sqrt{1-\bar{\alpha}_t}\hat{v}_\theta(x_t, t),$$

$$\hat{\epsilon}_\theta = \sqrt{1-\bar{\alpha}_t} x_t + \sqrt{\bar{\alpha}_t}\hat{v}_\theta(x_t, t).$$

Thus, velocity prediction is only a different coordinate system for the same two-dimensional linear relationship between x_0, ϵ, and x_t.
A more score-matching-flavored option is to predict the score of $q(x_t|x_0)$:

$$s_t = \nabla_{x_t} \log q(x_t|x_0) = -\frac{\epsilon}{\sqrt{1-\bar{\alpha}_t}}.$$

From a score prediction $\hat{s}_\theta(x_t, t)$, we can recover

$$\hat{\epsilon}_\theta = -\sqrt{1-\bar{\alpha}_t}\hat{s}_\theta(x_t, t).$$

Therefore, x_0-prediction, velocity-prediction, and score-prediction can all be converted back into the noise prediction required by the DDPM reverse step. They differ in the supervised target and scaling seen during training, not in whether the reverse process can be mathematically formed.

(b) [10 pts] Now implement your derived parametrization and train a model with the same configuration with your previous DDPM implementation with the only difference being the model parametrization. Include a grid of 16 samples and report your KID below. (Budget tip: You can compare the two parameterizations using shorter training runs, as long as they provide meaningful comparisons.)

Implementation status before the full Colab run: the DDPM method has been extended so that the same U-Net architecture can be trained with four prediction targets: ϵ, x_0, v, and score. The configs for the four runs keep the dataset, model architecture, optimizer, EMA, noise schedule, batch size, and number of iterations fixed; the only intended algorithmic change is `ddpm.prediction_type`.
The full experimental results for this question are pending the Part IV Colab sweep. The runner will write the final sample grid and KID table to:

`figures/q6_final_samples.png`, `figures/q6_kid_table.csv`.

After the run completes, this answer should be replaced with the actual 16 sample grids and the KID mean/std for each successful parameterization. No KID numbers are reported here until the evaluation has actually been run.

(c) [3 pts] Compare the performance of your new parametrization with that of the original parametrization. Share your findings below.

The comparison will use ϵ-prediction as the baseline and compare the alternative parameterizations using three kinds of evidence: final KID mean/std, qualitative samples, and training diagnostics. The training script now logs both the direct parameterization loss and comparable quantities such as noise MSE, x_0 MSE, cosine similarity with the true noise, and timestep-bin noise MSE. This matters because the raw training loss is not directly comparable across ϵ, x_0, v, and score targets.
This comparison is pending the full Colab sweep. The final answer should cite `figures/q6_loss_curves.png`, `figures/q6_noise_mse_curves.png`, `figures/q6_timestep_bin_heatmap.png`, and the KID table generated from the completed runs.

Q7. Sampling Steps Ablation [10 pts]

(a) [10 pts] DDPM typically uses 1000 timesteps for sampling, which is slow. How about using fewer steps? Report the KID scores for DDPM sampling algorithm with $100, 300, 500, 700, 900$ steps and compare with your original sampling with 1000 steps. Include 1 sample for each number of steps to qualitatively show your comparisons as well.

Implementation status before evaluation: the sampler now supports both the original full 1000-step stochastic DDPM reverse chain and a deterministic strided sampler for fewer steps. For $N < 1000$, the sampler selects N descending timesteps from 999 to 0. At each selected

timestep it predicts $\hat{\epsilon}_\theta$, reconstructs

$$\hat{x}_0 = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\hat{\epsilon}_\theta}{\sqrt{\bar{\alpha}_t}},$$

and jumps to the next selected timestep s with

$$x_s = \sqrt{\bar{\alpha}_s}\hat{x}_0 + \sqrt{1-\bar{\alpha}_s}\hat{\epsilon}_\theta.$$

Thus, the fewer-step sampler is a deterministic strided DDIM-style sampler using the DDPM-trained predictor, rather than the original adjacent-step posterior applied 1000 times.
The KID table and sample comparison for $100, 300, 500, 700, 900, 1000$ steps are pending the Colab evaluation run. The runner will write the required outputs to `figures/q7_kid_table.csv`, `figures/q7_kid_vs_steps.png`, and `figures/q7_sampling_steps_samples.png`. No KID values are reported until those files have been generated from an actual run.

Part V: Reflection (10 points)

Now that you have had a fully immersive experience with DDPM, let's take a step back and reflect on what we have learned so far.

Q8. Reflection Questions **[10 pts]**

(a) [3 pts] Based on your experience implementing and experimenting with DDPM: What do you think DDPM is bad at? Identify 2-3 limitations you observed or suspect. These could relate to sample quality, training efficiency, sampling speed, controllability, or anything else. Be specificconnect each limitation to something concrete from your experiments or implementation.

(b) [3 pts] For those limitations you identified, how do you think they could be improved? Propose at least 2 concrete ideas. These don't need to be novelthey can be things you've heard about, ideas from papers, or your own speculation. For each idea, briefly explain why you think it might help. (We're not asking you to implement thesejust to think critically about the design space.)

(c) [3 pts] What ablations or experiments are you still curious about? List 1-2 things you would explore if you had more time and compute budget. Why do these interest you?

(d) [1 pts] List all the resources that you found helpful for this homework. Include AI tools, open source code, tutorials, papers, classmates that helped you, etc.

That's it! You have completed HW 1! Congrats on your freshly baked diffusion models!

References

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. In *Proceedings of International Conference on Computer Vision (ICCV)*, December 2015.

Anton Obukhov, Maximilian Seitzer, Po-Wei Wu, Semen Zhydenko, Jonathan Kyl, and Elvis Yu-Jing Lin. High-fidelity performance metrics for generative models in pytorch, 2020. URL https://github.com/toshas/torch-fidelity. Version: 0.3.0, DOI: 10.5281/zenodo.4957738.

Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.